Three Stage Dynamic Context Engineering for LLM Tool Usage

Sai Guntabonu
Georgia Institute of Technology
sguntabonu3@gatech.edu

Shivansh Sharma
Georgia Institute of Technology
ssharma667@gatech.edu

Pranav Tadepalli
Georgia Institute of Technology
pt@gatech.edu

Agam Saraf
Georgia Institute of Technology
asaraf61@gatech.edu

Abstract

Tool-calling performance in large language models (LLMs) remains brittle, often failing due to subtle prompt-sensitivity or recurring model-specific errors. We study an application of Agentic Context Engineering (ACE)—an iterative Generator → Reflector → Curator loop—which is optimized for creating tool-specific memory (Zhang et al., 2025). Using DeepSeek-V3 and the Berkeley Function-Calling benchmark (Patil et al., 2025a), we implement an ACE-inspired pipeline that constructs a playbook of tool-specific guidance from a seeded learning split. During the playbook training phase, the Generator produces tool calls without any playbook; the Reflector analyzes deviations from ground truth; and the Curator converts these reflections into add/modify/remove operations on a JSON playbook organized by tool group. We further evaluate the implications of replacing the curator and reflector with smaller models with lower reasoning capability. At evaluation time, the original model is evaluated with either the full playbook prepended or with dynamically retrieved entries selected through tool calls. Across various pipeline variations, we find that our approach boosts tool-calling accuracy by about 10% through highly specialized and dynamically managed memory.

1 Introduction

Large language models (LLMs) increasingly rely on structured function-calling to interface with external tools, APIs, and software systems. However, despite improvements in model reasoning and instruction-following, tool-use performance remains brittle: models repeatedly make the same mistakes, exhibit high prompt sensitivity, and struggle to generalize tool behavior across long sequences of interactions. A core limitation is memory management. Existing approaches either rely on extremely long contexts—which quickly become unwieldy—or on coarse summarization modules that degrade important procedural details. These methods are not optimized for environments where dozens of heterogeneous tools each require precise, tool-specific behavioral constraints. As a result, models lack a scalable mechanism for accumulating, refining, and reusing structured knowledge about how individual tools should be used.

Agentic Context Engineering (ACE) (Zhang et al., 2025) was introduced to address long-term memory and error-driven learning for general reasoning tasks. ACE iteratively applies a Generator → Reflector → Curator loop to extract insights from failures and inject them back into the model's future context. While promising, prior ACE systems maintain a single, monolithic memory, making it difficult to separate behaviors across tools or to maintain fine-grained guidance for APIs with distinct parameters, constraints, and affordances.

We propose an ACE adaptation tailored specifically to LLM tool-use called Dynamic Context Engineering (DCE). Our key idea is to treat the playbook as a structured memory database, where each tool group has its own dedicated section. The Curator acts directly on the playbook as a database agent, issuing explicit `ADD`, `MODIFY`, and `REMOVE` operations rather than relying on purely additive updates or lossy summarization. This modular design maintains hyper-specific, non-interfering guidance for each tool, enabling the model to correct recurring errors without inflating context length. Further, our pipeline incorporates any available signals such as ground truth, execution traces, and state of environment into the Reflector and Curator modules, yielding highly targeted feedback on failure cases.

To study the effects of this architecture, we implement the full DCE pipeline around DeepSeek-V3 and extend the Berkeley Function-Calling Leaderboard (BFCL) (Patil et al., 2025a) benchmark codebase to support dataset splitting, playbook construction, dynamic tool-querying, and

model-swappable Reflector/Curator components. During training, the model receives no playbook; its outputs and errors drive reflection and curated memory updates. During evaluation, the same underlying model is conditioned either on the entire playbook or only on entries it dynamically retrieves via tool calls.

Our contributions are threefold:

1. A tool-specific framework that replaces monolithic memory with structured, per-tool playbooks and curator-driven database operations.

2. A training pipeline that integrates ground-truth tool calls and evaluation traces, enabling highly targeted reflections and fine-grained memory updates.

3. A comprehensive empirical study comparing baselines, full-playbook prompting, dynamic tool-querying, and stronger reflector/curator models, demonstrating a new method of memory management and offering insight into using smaller models to curate said memory.

This work provides a systematic investigation of ACE for programmatic tool use through a new implementation of the memory module and establishes a foundation for scalable, self-improving tool-calling behavior in LLMs.

2 Background and Related Work

2.1 Formalizing Tool Calling

We formalize tool calling as an interaction between a language model and a set of externally implemented functions. Let $\mathcal{T} = \{t_1, \ldots, t_m\}$ denote a finite set of tools, where each tool t is specified by a name, a structured argument schema (e.g., JSON fields with types and constraints), and a return type. Given a natural-language query x and a tool inventory $\mathcal{T}$, the model's objective is to output a structured tool call $c = (t, a)$, where $t \in \mathcal{T}$ and a is a well-typed argument object that satisfies the schema of t. In benchmarked function-calling settings, each example provides the user prompt, the set of available tools (or tool group), and a ground-truth call $c^\star$, and the model is evaluated on whether it selects the correct tool name and arguments (possibly up to value- or schema-level equivalence). In multi-turn agent settings, tool calling is embedded in a broader dialogue between the model and an environment. The model alternates between (i) emitting tool calls that are executed by a tool backend (simulated APIs, program interpreters, file systems), and (ii) receiving observations that describe the tool outputs, which are then incorporated into subsequent context.

2.2 Reflexion

Reflexion (Shinn et al., 2023) is a framework for verbal reinforcement learning that improves language agents not by updating model weights, but by generating natural language feedback and storing it as episodic memory. Concretely, Reflexion instantiates three roles: an *Actor* M_a that produces actions or tool calls given the current state; an *Evaluator* M_e (or environment-based scoring function) that converts trajectories and outcomes into scalar or binary feedback; and a *Self-Reflection* model M_{sr} that transforms this feedback plus the trajectory into textual "experience summaries." These summaries are appended to a long-term memory buffer and fed back into the Actor's prompt in subsequent episodes, serving as a semantic gradient that nudges the policy toward better decisions.

Reflexion has been shown to yield substantial gains across decision-making, multi-hop reasoning, and code generation tasks by enabling agents to learn from sparse environment feedback without gradient-based reinforcement learning or fine-tuning. However, Reflexion maintains a single, monolithic memory buffer shared across tasks and tools, and its evaluator returns simple scalar or binary success signals.

2.3 ACE

Agentic Context Engineering (ACE) generalizes this idea of context-based adaptation into a modular framework that treats prompts as evolving *playbooks* rather than static instructions or short summaries (Zhang et al., 2025). ACE is designed for both offline prompt optimization (e.g., system prompts) and online memory adaptation (e.g., agent histories), with the key goal of avoiding two failure modes of prior context methods: brevity bias, where optimized prompts collapse into overly generic instructions that drop domain-specific heuristics, and context collapse, where repeated rewriting erodes accumulated detail over time. Instead of monolithic rewriting, ACE represents context as a collection of structured "bullets" and applies small, incremental "delta" updates that add entries while preserving the rest of the playbook.

Architecturally, ACE again uses a three-role decomposition: a *Generator* that produces reasoning trajectories or tool calls for new queries; a *Reflector* that analyzes successes and failures to extract concrete lessons; and a *Curator* that integrates those lessons as localized edits to the playbook. The resulting contexts can grow into detailed, domain-specific playbooks that accumulate reusable strategies, with periodic "grow-and-refine" steps to deduplicate and compress when necessary. Empirically, ACE improves over strong prompt-optimization and memory baselines on both agent benchmarks and domain-specific reasoning tasks, while also reducing adaptation cost and latency. However, the ACE memory module is monolithic and thus can grow large overtime as new deltas are added, with no ability to delete and modify existing deltas.

3 Method

3.1 Problem

LLM tool calling is inherently a modular reasoning problem: each tool has its own API surface, parameter constraints, and failure modes, yet existing memory-based approaches—including Reflexion-style episodic memory and the original ACE framework—maintain a single monolithic memory store. This structure implicitly assumes that all insights are globally relevant, even though tool-use errors are highly localized. As the number of tools grows, monolithic memories intermingle unrelated guidance, create contextual clutter, and degrade retrieval precision. Memory also grows rapidly: even with de-duplication mechanisms such as ACE's "grow-and-refine" cycle (Zhang et al., 2025), large blocks of heterogeneous insights accumulate many near-duplicate or subtly divergent rules. This inflates context length, increases redundancy, and imposes a heavy cognitive and computational burden on the model during inference.

Injecting the entire accumulated memory into the prompt further exacerbates scalability issues, especially as tool ecosystems expand into dozens or hundreds of callable functions. Full-memory prompting increases token cost, adds distractor content, and obscures the information most relevant to the current tool call. These limitations motivate the need for a selectively retrievable memory system that surfaces only the insights pertinent to a given tool, while still preserving the fidelity necessary for reliable tool-use behavior. The core challenge we address is therefore how to design a memory architecture that remains modular, compact, and adaptively retrievable—without sacrificing accuracy or the fine-grained behavioral guidance required for robust tool calling.

3.2 Our Solution

We introduce Dynamic Context Engineering (DCE), a structured memory architecture centered around the playbook. The playbook is a persistent, centralized memory module that stores tool-use knowledge accumulated during training. Unlike prior systems that maintain a single, undifferentiated memory buffer, the playbook is explicitly partitioned into sections, one per tool group. Each section contains a set of curated "deltas"—concise statements that encode correct usage patterns, common failure modes, and tool-specific constraints. This design aligns the memory structure with the inherent modularity of tool calling, enabling the system to store and retrieve guidance precisely at the level of abstraction where errors occur.

Our solution provides a more expressive and adaptive memory-update mechanism through an agentic Curator. Instead of only appending new reflections to memory, the Curator issues explicit `ADD`, `DELETE`, and `MODIFY` operations on individual playbook entries. This transforms the playbook from a passive log of prior reflections into an actively maintained database. By allowing the Curator to revise, refine, or remove outdated insights, the system can correct earlier misconceptions, prevent memory bloat, and converge toward stable, high-quality guidance for each tool group. These localized updates also reduce the risk of interference across tools and mitigate the long-term drift seen in monolithic memory systems.

Finally, the architecture supports both global and selective retrieval modes during evaluation. In the full-playbook setting, all sections are prepended to the model's context, ensuring maximum recall of learned guidance but incurring higher context cost. To improve scalability, we introduce a selective-retrieval mechanism in which the model retrieves only the relevant tool-group sections via a tool call before generating the final tool-use prediction. This dynamic-query approach reduces unnecessary context load while preserving accuracy. Together, these components provide a modular, adaptable, and scalable memory system tailored specifically to the demands of LLM tool calling.

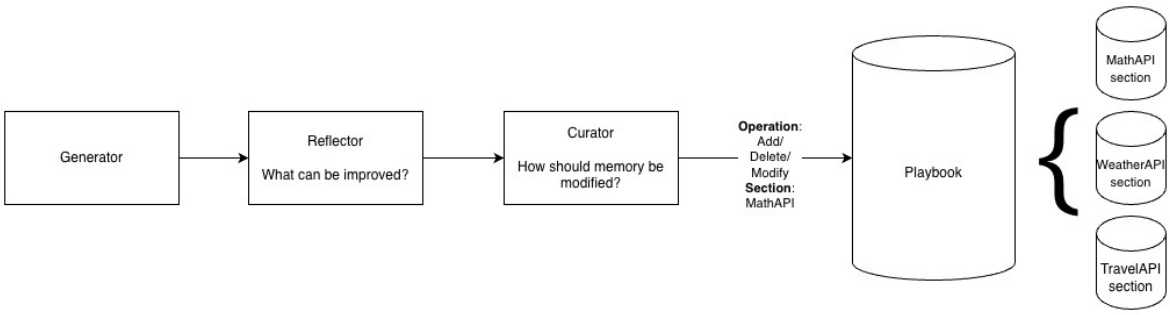

Figure 1: Learning Phase

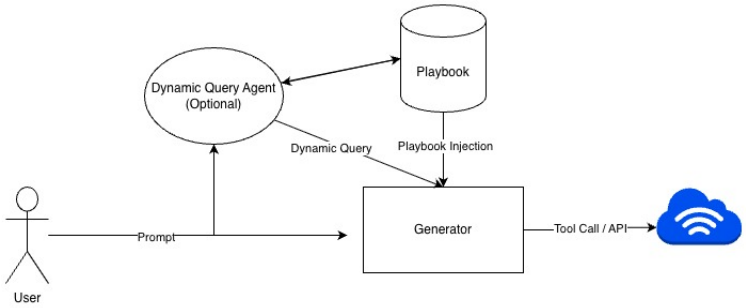

Figure 2: Eval Phase

3.3 Architecture

3.3.1 Learning Phase

The learning phase adapts the ACE framework to the structured tool-calling setting by iterating a Generator → Reflector → Curator loop over the training split (Figure 1). As in the original ACE formulation, the Generator produces an initial tool call using only the raw benchmark prompt, without any playbook guidance. Its output is executed against the BFCL tool simulator to obtain ground-truth feedback about correctness. The Reflector receives the prompt, the model's predicted tool call, the execution trace, and the gold tool invocation. It generates a structured natural-language reflection, identifying the error type, root cause, and corrective strategy.

Unlike the original ACE system, where all reflections are added to a single growing memory block, our adaptation introduces a partitioned playbook divided into sections corresponding to tool groups (e.g., MathAPI, WeatherAPI). The Curator acts as an agent over this structured memory, deciding which tool-group section the reflection applies to and proposing an explicit operation: `ADD` a new insight, `MODIFY` an existing entry, or `DELETE` an outdated insight. This expanded set of operations enables fine-grained memory revision rather than the append-only updates used in the original ACE system, allowing the playbook to preserve the accuracy of existing entries. Each Curator decision produces a structured database update, which is immediately applied to the appropriate playbook section. By the end of training, the system has constructed a tool-group–indexed playbook that reflects thousands of model-specific failure modes and their corresponding corrections.

3.3.2 Evaluation Phase

During the evaluation phase, no further learning or playbook modification occurs; the Generator operates solely with the fixed playbook produced during the learning phase. We evaluate two variations (Figure 2).

In the *Full Playbook* condition, the entire playbook—across all tool groups—is prepended to the Generator's context along with the user prompt. This condition measures the upper bound of improvement when all accumulated guidance is made explicitly available at inference time, at the cost of potentially large context overhead.

In the *Dynamic Query* condition, we introduce a separate LLM agent that receives the user prompt and issues a "generous" retrieval query to the playbook. Its goal is to identify a subset of the tool-

group sections plausibly relevant to the upcoming tool call. Based on this query, only those selected sections are returned and injected into the Generator's prompt. The Generator then produces the final tool call using a reduced subset of the playbook. This mode tests whether selective retrieval can retain most of the benefits of full-playbook prompting while avoiding unnecessary context expansion.

3.4 Data

We use the Berkeley Function Calling Leaderboard (BFCL) benchmark for both the learning and evaluation phases (Patil et al., 2025a). BFCL provides a large, diverse collection of single-turn, multi-turn, crowd-sourced, and agentic function-calling tasks, spanning real APIs, Python/Java/JavaScript functions, and stateful simulated environments. Each example contains a natural-language prompt and a structured ground-truth function call. BFCL evaluates predictions through deterministic Abstract Syntax Tree (AST) matching, which verifies function names and parameter values without requiring execution, and through execution-based checks for subsets of functions. This combination yields scalable and precise evaluation of tool-call correctness, making BFCL well-suited for studying tool-use reliability, error patterns, and generalization under both short-horizon and long-horizon interactions.

4 Experiments

4.1 Data Preparation

Our experiments follow the BFCLv3 benchmark standard rather than the latest BFCLv4. We choose the BFCLv3 standard since it does not contain tasks requiring Serp Web API which comes with a high cost subscription and also does not require memory API which maintains dependencies between separate data points, making sampling difficult. To create our learning and evaluation sets, we took two exclusive samples from BFCL data. This sample was necessary to ensure reasonable training costs (sampling reduced our total costs to $50 dollars from the estimated $150 had we covered the entire dataset due to the cost of Serp Web API and DeepSeek and Claude API credits). We sampled the data such that the original distribution of categories from BFCL is kept intact as shown in Table 1. Our learning set consists of 1866 samples and the evaluation set consists of 934 samples (roughly 2-to-1 train-test split). Using the same groupings as BFCL's leaderboard, we group tasks into Single (live and non-live), Multi-Turn, Relevance, and Irrelevance. Single only requires one API response from the LLM and is self-contained. Multi-Turn involves multiple interactions and API calls where the LLM must remember context from prior interactions. Relevance and irrelevance assess the model's ability to recognize when a provided tool is appropriate for the task and when the task should be completed without any tool call. Live data is real-world user data, while non-live data is generated by researchers.

Category	Train	Test
irrelevance	160	80
live_irrelevance	159	79
live_multiple	159	79
live_parallel	11	5
live_parallel_multiple	16	8
live_relevance	11	5
live_simple	159	79
multi_turn_base	133	67
multi_turn_long_context	133	67
multi_turn_miss_func	133	67
multi_turn_miss_param	133	67
multiple	133	67
parallel	133	67
parallel_multiple	133	67
simple_java	67	33
simple_javascript	33	17
simple_python	160	80

Table 1: Sampled dataset size per category.

4.2 Baseline

For the baseline, we evaluate the unmodified DeepSeek-V3.2-Exp-FC model, a 671B-parameter function-calling model (DeepSeek-AI, 2025), on the held-out test split of 934 BFCL examples. No memory or ACE components are used in this condition: the model receives only the raw benchmark prompt without any playbook, reflection, or curated guidance. We run the model in deterministic mode with a temperature of 0.001 to minimize sampling variance, and evaluate its outputs using the BFCL tool-execution and AST-matching criteria. This baseline reflects the model's native tool-calling ability and provides a reference point against which we measure improvements introduced by DCE training, playbook construction, and selective retrieval.

4.3 DCE

4.3.1 Generator

The Generator follows the same DeepSeek-V3.2-Exp-FC model from the baseline to produce tool calls throughout both learning and evaluation. In the learning phase, the Generator is intentionally

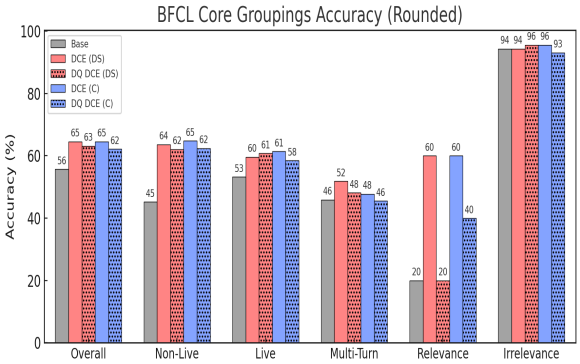

Figure 3: DCE Core Groupings Accuracy %
DS = DeepSeek, C = Claude, DQ = Dynamic Query

run *without* access to the playbook so that its raw mistakes can drive the Reflector and Curator updates. In the evaluation phase, the Generator is applied to the held-out BFCL test split, this time with access to the completed playbook produced during training. We report results for two inference settings: one in which the full playbook is injected into the context, and another using our dynamic filtering mechanism in section 4.4. This allows us to isolate the impact of memory assistance and selective retrieval on tool-call accuracy across the diverse categories of the BFCL benchmark. Figure 3 reports Generator performance across the BFCL task groupings and shows that introducing DCE yields improvements in nearly every category. The largest gains occur on Relevance tasks, where the model must correctly avoid issuing any tool call when the query can be answered directly. By contrast, Irrelevance tasks—which require refraining from calling a tool when none of the options are useful—show little change, likely because baseline performance was already high. Table 2 presents results disaggregated by major API families. Here, DCE with DeepSeek improves over the baseline for nearly all APIs, with the notable exceptions of the Math and Trading Bot APIs, in which all accuracies were already substantially higher than in other domains. These patterns suggest that tasks with lower intrinsic difficulty or more rigid parameter schemas benefit less from additional memory, whereas more semantically complex or error-prone APIs exhibit clearer gains from DCE's structured guidance.

4.3.2 Reflector

The Reflector operates exclusively during the learning phase and is responsible for producing structured natural-language reflections that diagnose the Generator's errors. Because the Reflector does not issue function calls itself, we use the non-function-calling DeepSeek-V3.2-Exp variant in "non-thinking" mode to reduce computational overhead. To assess the extent to which the Reflector's reasoning capability influences the quality of the resulting playbook, we experiment with replacing it with the smaller Claude 4.5 Haiku model in Section 4.4. This model swap enables us to isolate whether stronger reflective reasoning leads to more useful insights—and ultimately a more effective playbook—or whether the reflection step is robust to reduced model capacity.

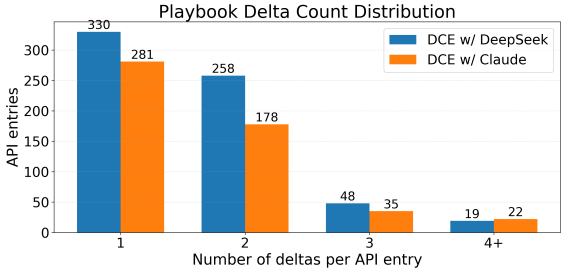

Figure 4: Number of "delta" notes left for each API function when the curator is Claude vs Deepseek

4.3.3 Curator

The Curator is also active only during the learning phase and receives both the Generator's predictions and the Reflector's analyses. Using the same DeepSeek-V3.2-Exp model, the Curator outputs structured operations (`ADD`, `DELETE`, or `MODIFY`) targeting specific tool-group sections of the playbook. These fine-grained edit operations differentiate our system from prior ACE implementations and allow the playbook to remain compact, relevant, and precisely scoped. Across learning, we logged Curator behaviors, including the frequency of each operation type and the total number of entries accumulated in the playbook. These metrics allow us to analyze how different curator models shape playbook

Table 2: DCE API Category Accuracy (%)

Category	Base	DCE (DS)	DCE (C)	DQ DCE (C)	DQ DCE (DS)
Vehicle Control	43.40	47.17	39.62	32.08	**49.06**
Trading Bots	**67.57**	66.22	**67.57**	66.22	63.51
Travel Booking	28.79	**43.94**	42.42	39.39	34.85
Gorilla File Sys	41.33	**48.00**	38.67	40.00	44.00
Message API	**59.26**	**59.26**	51.85	48.15	55.56
Twitter API	34.00	**44.00**	**44.00**	40.00	38.00
Ticket API	25.00	**37.50**	29.17	22.92	27.08
Math API	61.11	61.11	52.78	58.33	**63.89**

Key: Base = Baseline; DCE = Dynamic Context Engineering; DQ = Dynamic Query; DS = DeepSeek; C = Claude

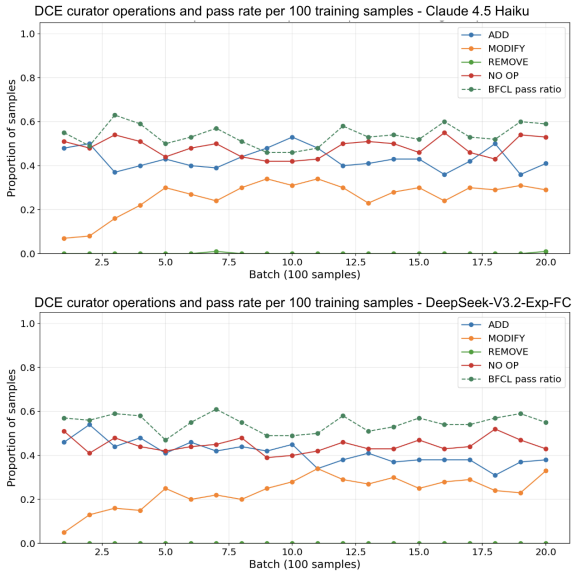

Figure 5: Curator operations during learning phase

curation. As shown in Figure 4, the DeepSeek-based curator produces a larger number of deltas than Claude. Figure 5 indicates that both curators increasingly favor `MODIFY` operations as the playbook matures—consistent with a regime in which new insights become rarer and refinement increases. Notably, both variants rarely issue `DELETE` operations, suggesting a reluctance to remove entries even when their relevance diminishes. This conservative deletion behavior highlights a potential need for more explicit prompting or auxiliary signals (e.g., size thresholds) to encourage pruning when the playbook grows large.

4.4 Weak Reflector and Curator

To assess the sensitivity of DCE to the strength of its auxiliary reasoning modules, we replaced both the Reflector and Curator with Claude-Haiku-4-5-20251001 (Prompt), a smaller model whose chat variant achieves 26.24% on the full BFCL leaderboard—far below the 47.76% achieved by DeepSeek-V3.2-Exp (Prompt) (Patil et al., 2025b). In this setting, only the Generator remains fixed as DeepSeek-V3.2-Exp-FC, ensuring that any changes in performance arise solely from weakened reasoning from reflection and curation. Despite the sizable gap between the two models' standalone BFCL accuracies, we observe near identical performance in downstream tool-calling performance when using Haiku to construct the playbook (Table 2). Figure 6 illustrates one example where the playbook entries produced by the Claude curator are similar in nature to the ones produced by a

Example Playbook Entry with Claude-4.5-Haiku Curator
event_settings_api
→ `delta1: Restore/bring-back requests → use restore_mobile_app_alert_config, not find_* lookups`
→ `delta2: Action verbs (restore, bring back, recover) map to write ops, not read/query ops`

Example Playbook Entry with DeepSeek-V3.2-Exp Curator
event_settings_api
→ `delta1: Use restore_mobile_app_alert_config for restoration operations, not find functions.`
→ `delta2: Use get_event_specification_infos_by_ids for unified event specification lookup, not separate built-in/custom functions.`

Figure 6: Playbook API entry examples with large and smaller curator models

DeepSeek curator. Notably, the playbook produced by the Haiku modules still enables the stronger DeepSeek Generator to outperform its baseline accuracy. This result suggests that DCE is robust to small reductions in Reflector/Curator capabilities and that even comparatively weak models can produce useful, structured guidance that a stronger Generator can exploit effectively—highlighting an appealing asymmetry in which smaller models potentially bootstrap more capable ones through curated memory.

4.5 Dynamic Query

Dynamic Query extends the DCE inference pipeline by modifying how playbook information is surfaced to the Generator. Instead of injecting the entire playbook into the prompt, we introduce a separate DeepSeek-V3.2-Exp module responsible for selecting which tool-group sections to retrieve. This module receives both the user prompt and high-level descriptions of all available playbook sections, and is instructed to be "generous" in selecting any sections that might plausibly assist the Generator. The resulting subset of sections is then appended to the Generator's input, preserving the same decoding setup used in the base DCE evaluation. We quantify how accuracy changes when the Generator operates with selectively filtered memory rather than full-playbook access. Although the playbook remained well within DeepSeek's context window across all experiments, dynamic querying simulates more realistic deployments where memory grows large or retrieval must be bandwidth-aware. Figure 7 shows that Dynamic Query becomes more selective as the number of available tools grows, even without any explicit guidance in the retriever's prompting. Empirically, we observe slight performance degradation relative to full-playbook DCE, with the primary exception oc-

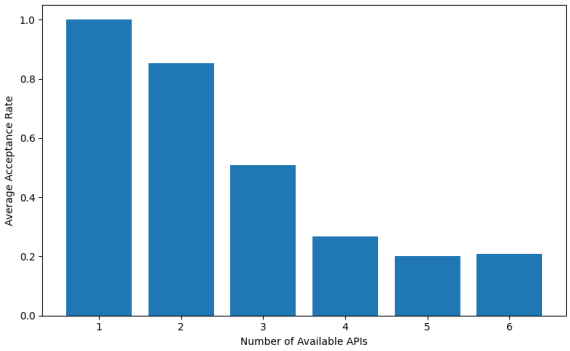

Figure 7: Dynamic Query Agent Tool Acceptance Rate by Available API Count (DeepSeek Playbook)

curring in the irrelevance category, where broader context appears more beneficial (Figure 3). Overall, dynamic querying offers a scalable alternative to full memory injection with limited performance cost.

5 Discussion

DCE provides consistent and robust gains over the baseline DeepSeek-V3.2 model, demonstrating that a single playbook-construction pass is sufficient to yield durable improvements in tool-calling reliability. Across BFCL's composite task groupings, DCE improves accuracy in nearly every category, even under long-context, multi-turn conditions where added memory risks distracting the model. Particularly notable is the 40-point gain in relevance detection, where the model must refrain from calling any tool when none is appropriate—an error mode that baseline LLMs frequently struggle with. At the tool level, DCE yields large improvements on domains such as Math and Twitter API, while producing minimal change on Message and Trading Bot APIs, suggesting that some domains benefit more clearly from structured memory than others. Evaluation across all three components of the architecture further underscores this robustness. Despite Claude 4.5 Haiku performing substantially worse than DeepSeek on BFCL as a standalone model, substituting Haiku for the Reflector and Curator produces only minor degradation in downstream accuracy. This indicates that reflection and curation need not be performed by the strongest available model and that inexpensive models can effectively maintain and update memory for stronger generators. Curator-operation logs also reveal that MODIFY operations are rare at the start but increase as the playbook matures, while DELETE operations remain rare, implying that the curator remains hesitant to remove stale entries. This behavior points to future opportunities for prompting refinements or additional signals—such as context-budget awareness—to encourage more aggressive DELETE operations.

The dynamic-query mechanism addresses a central limitation of long-term memory systems: unbounded growth in context length. Instead of injecting the entire playbook, dynamic query retrieves only the tool-group sections deemed potentially relevant by a separate agent. Across BFCL categories, this selective-retrieval setup preserves most of the gains achieved by full-playbook conditioning, and in some categories exhibits only marginal differences from the best DCE scores. Furthermore, we observed that Dynamic Query becomes increasingly selective as the number of available tools grows. Although the playbook in our experiments did not exceed the model's context window, the strong performance of dynamic querying suggests that it is a viable strategy for memory-scaling scenarios where the playbook may eventually outgrow available context. Future work should test retrieval under substantially larger playbooks and more realistic long-horizon agent deployments, where selective memory access is essential rather than optional.

6 Conclusion

This work introduces a structured, tool-partitioned adaptation of Dynamic Context Engineering designed specifically for LLM function calling. By treating accumulated guidance as an editable playbook rather than a monolithic memory buffer, our approach enables targeted error correction, controlled memory growth, and effective reuse of tool-specific insights. Experiments across BFCL show that this design reliably improves tool-use accuracy and remains resilient even when reflections and curation are produced by weaker models. The Dynamic Query mechanism further demonstrates that selective retrieval can provide most of the benefit of full-playbook prompting while reducing context size, pointing toward a practical path for scaling memory-assisted tool use. Overall, these results highlight the value of modular, editable, and retrieval-aware memory systems for building more reliable and adaptable tool-using language models.

7 Limitations

Our evaluation is constrained by several practical and methodological limitations. First, due to

API cost and compute requirements, experiments were conducted on a representative subset of BFCL rather than the full benchmark, which may limit coverage of rarer tools and long-tail failure modes. Second, our methodology relies on access to high-fidelity ground-truth tool calls and execution traces, whereas real-world systems often provide only partial or noisy environment feedback; future work should assess whether DCE maintains efficacy under weaker supervision, as suggested by prior ACE results (Zhang et al., 2025). Third, our experiments did not induce playbook sizes large enough to exceed model context limits, leaving open the question of how well dynamic querying performs under truly large-scale memory growth. Finally, the Reflector and Curator were evaluated using non-reasoning variants of DeepSeek and Claude; exploring reasoning-enabled or chain-of-thought models for these components may reveal stronger or qualitatively different playbook dynamics.

References

DeepSeek-AI. 2025. Deepseek-v3.2-exp model card. https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp. Accessed: 2025-11-19.

Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025a. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025b. Berkeley function calling leaderboard. https://gorilla.cs.berkeley.edu/leaderboard.html. Accessed: 2025-11-18.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc.

Yuchen Zhang, Alexis Chevalier, Anirudh Ajith, and Danqi Chen. 2025. Agentic context engineering (ace): Evolving playbooks for self-improving llms. *arXiv preprint arXiv:2510.04618*.